OOP Week 6 - Exception Handling in Java

Object Oriented Programming with Java — Week 6

> **The story in one line:** Every program is a delivery driver. Most days the roads are clear, but sometimes there is a roadblock. Exception handling is the driver's plan for what to do when the road is closed, instead of abandoning the van in the middle of the highway.

Index

- 1. What Is an Exception? — *When your program stops being a polite guest and throws a tantrum.*
- 2. Life Without Exception Handling — *A crash course, quite literally.*
- 3. Why Handle Exceptions at All? — *Four good reasons to stop crashing.*
- 4. Types of Exceptions in Java — *Checked, unchecked and errors: the three kinds of trouble.*
- 5. The Exception Hierarchy — *The family tree, with Throwable as grandparent.*
- 6. try, catch and finally — *Attempt, rescue, clean up.*
- 7. Multiple catch Blocks — *One try, many safety nets.*
- 8. Nested try Blocks — *Safety nets inside safety nets.*
- 9. The throw Keyword — *Raising the alarm on purpose.*
- 10. The throws Keyword — *Putting a warning label on a method.*
- 11. throw vs throws — *Two letters, two very different jobs.*
- 12. User-Defined (Custom) Exceptions — *Invent your own kind of trouble.*
- 13. Summary — *The whole chapter in five bullets.*
- 14. Practice Exercises — *Your homework, with hints.*
- 15. Think-It-Through Questions — *Five questions that will not let you coast.*

Learning Objectives

This chapter covers the first four objectives. The remaining three (file handling with streams, text and binary files, serialization) are covered in the next lecture on file I/O. Exceptions come first because file operations are one of the most common sources of exceptions: files go missing, disks fill up, and permissions are denied.

The same three-step idea (`try` , `catch` , `finally`) exists in other object oriented languages such as C++ and C#, so the concepts transfer directly.

1. What Is an Exception?

Up to now, every program in this course has followed a *normal flow*: we created classes, created objects, and used inheritance, polymorphism and abstract classes, and the program ran from the first line to the last.

> **An exception is an abnormal condition or unexpected event that occurs during program execution and disrupts the normal flow of a program.**

It is called *abnormal* precisely because it breaks the normal flow.

Typical Examples

- Dividing a number by zero
- Accessing an invalid array index (for example `a[0], a[1], a[2], a[3]` exist but you ask for `a[10]`)
- Opening a file that does not exist
- Invalid user input (the program expects an `int` and the user types something else)

Mathematically, division by zero is undefined. In a program there is no sensible number to return, so Java signals an error instead.

The Big Picture

When the program meets one of these situations there are exactly two outcomes:

Situation	Result
No exception	The code runs properly and successfully, as the client requirement intended
Exception occurs	We must *handle* it through an implementation, otherwise the program dies

Handling the exception is done with three blocks: **try** (the risky code), **catch** (the exception type and its handler) and **finally** (the clean-up code).

2. Life Without Exception Handling

Here is a program with no protection at all:

```
class Demo {
  public static void main(String args[]) {
    int a = 10;
    int b = 0;

    int result = a / b;

    System.out.println(result);
  }
}
```

Output

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
Program terminates unexpectedly.
```

What happened, line by line:

1. `a` and `b` are initialised without trouble.
2. `a / b` is `10 / 0`, which is illegal for integers. Java creates an `ArithmeticException` and the normal flow stops there.
3. `System.out.println(result);` is **never executed**. The program is terminated unexpectedly.

The program was syntactically correct. It compiled fine and failed only while running. That is why we need a mechanism that deals with *runtime errors*.

3. Why Handle Exceptions at All?

> **Exception handling allows a program to handle runtime errors gracefully and continue execution.**

We add exception handling as an extra feature on top of our normal program. The advantages are:

Advantage	What it means in practice
Prevents abnormal program termination	A problem in the middle of execution no longer kills the whole program
Separates error-handling code from normal logic	The normal code and the error-handling code live in different blocks, so each stays readable
Improves program reliability	The program behaves predictably even when something goes wrong
Provides meaningful error messages	The user sees "Cannot divide by zero" or "Invalid array index" instead of a cryptic crash dump

> **Note for Review:** *Graceful* does not mean the error disappears. It means the error is **noticed, reported and contained**, and the program decides what happens next.

4. Types of Exceptions in Java

Java divides trouble into three main categories:

- **Checked Exceptions**
- **Unchecked Exceptions**
- **Errors**

4.1 Checked Exceptions

Checked exceptions are checked by the compiler during compilation. The programmer **must** handle them using `try-catch`, or declare them using `throws`. If you do neither, the code does not compile.

Examples

- `IOException`
- `SQLException`
- `FileNotFoundException`
- `ClassNotFoundException`

Example program

```
import java.io.*;

class Demo {
  public static void main(String args[]) {
    try {
      FileReader file = new FileReader("data.txt");
    }
    catch(FileNotFoundException e) {
      System.out.println("File not found");
    }
  }
}
```

How to read this program:

- `import java.io.*;` imports every class of the input/output package. It is needed because `FileReader` and `FileNotFoundException` live there.
- `FileReader` is a predefined class. We create an object of it for the file `data.txt`.
- If `data.txt` does not exist, a `FileNotFoundException` is raised and the `catch` block prints `File not found`.
- Because the compiler forced us to deal with this possibility, `FileNotFoundException` is a **checked** exception.

4.2 Unchecked Exceptions

Unchecked exceptions occur **during runtime** and are subclasses of `RuntimeException`. The compiler does not force you to handle them.

Examples

- `ArithmeticException`
- `NullPointerException`
- `ArrayIndexOutOfBoundsException`
- `NumberFormatException`

Example: ArithmeticException

```
class Demo {
  public static void main(String args[]) {
    int a = 10;
    int b = 0;

    System.out.println(a/b);
  }
}
```

Something divided by zero raises an `ArithmeticException`.

Example: ArrayIndexOutOfBoundsException

```
class Demo {
  public static void main(String args[]) {
    int arr[] = {10,20,30};

    System.out.println(arr[5]);
  }
}
```

The array has three elements (valid indices are `0`, `1`, `2`), but the program asks for `arr[5]`. That index is out of bounds, so Java raises an `ArrayIndexOutOfBoundsException`.

> **Note for Review:** Both programs compile perfectly. The compiler cannot know that `b` will be zero or that index `5` is invalid. These problems appear only when the program runs. That is exactly what "unchecked" means.

4.3 Errors

Errors represent **serious problems that usually cannot be handled by applications**.

Examples

- `OutOfMemoryError` (for example `java.lang.OutOfMemoryError`)
- `StackOverflowError`

An `OutOfMemoryError` occurs when memory is exhausted. A `StackOverflowError` occurs when the call stack becomes too deep, typically because of runaway recursion. These problems live in the JVM or system environment, not in your business logic, so a `catch` block rarely helps.

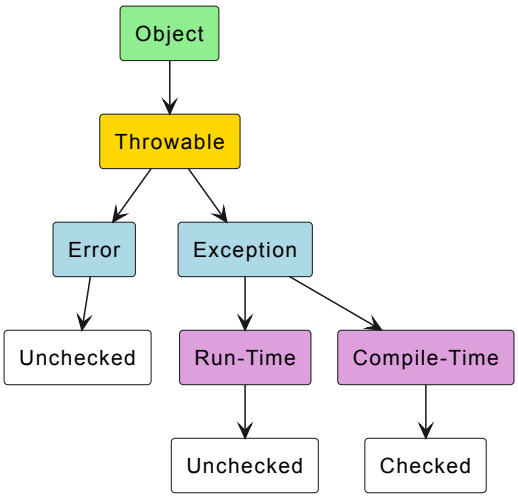
4.4 Side-by-Side Comparison

Feature	Checked	Unchecked	Error
When detected	Compile time	Runtime	Runtime
Must handle or declare?	Yes	No	No
Superclass	`Exception` (not `RuntimeException`)	`RuntimeException`	`Error`
Typical example	`FileNotFoundException`	`ArithmeticException`	`OutOfMemoryError`
Can the app recover?	Usually yes	Usually yes	Usually no

> **Pro-Tip:** Exams love the question "What is the difference between checked and unchecked exceptions?" The one-line answer is that the compiler **forces** you to deal with checked ones and does **not** force you to deal with unchecked ones.

5. The Exception Hierarchy

Everything that can be thrown in Java descends from `Object`, and then from `Throwable`. `Throwable` splits into `Error` and `Exception`. `Exception` in turn splits into the run-time group and the compile-time group.



Reading the tree:

- **Error** is unchecked.
- **Exception** → **Run-Time** is unchecked.
- **Exception** → **Compile-Time** is checked.

> **Note for Review:** "Compile-Time" in this diagram is shorthand for "checked at compile time". The exception itself still occurs while the program runs. What happens at compile time is the compiler's *check* that you have handled or declared it.

6. try, catch and finally

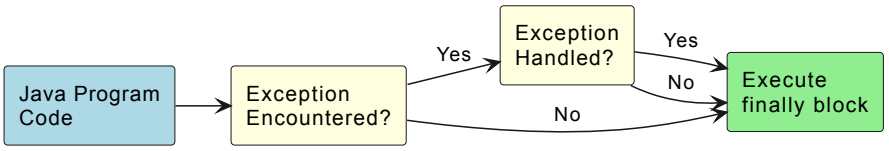
6.1 The Three Blocks

Block	Job
`try`	Contains the code that **may generate** an exception
`catch`	**Handles** the exception (you name the exception type it can handle)
`finally`	**Executes always**, whether an exception occurs or not (clean-up code)

An analogy. `try` is attempting a risky recipe. `catch` is the fire extinguisher you use if the pan catches fire. `finally` is washing the dishes: you do it whether or not the kitchen caught fire.

6.2 The Decision Flow

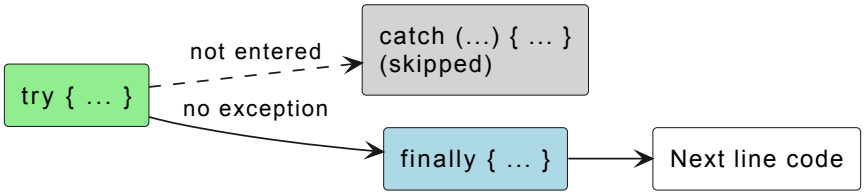
When the Java program runs, there are two questions to ask: was an exception encountered, and if so, was it handled? In *every* case the `finally` block runs.



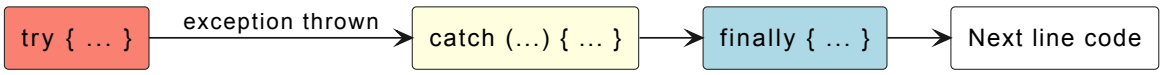
- **No exception:** straight to `finally`.
- **Exception, handled:** the `catch` block runs, then `finally`.
- **Exception, not handled:** `finally` still runs. (The exception then continues to propagate upward, which is why unhandled exceptions can still crash the program, but only after `finally` has done its clean-up.)

6.3 Without Exception vs With Exception

Without an exception, the `catch` block is skipped entirely. Control goes from `try` directly to `finally`, then to the next line of code. The `catch` block is not needed for this path to work.



With an exception, control moves from `try` to `catch`, then to `finally`, then to the next line of code.



6.4 A Complete Example

```
class Demo {
  public static void main(String args[]) {
    try {
      int result = 10/0;
      System.out.println(result);
    }
    catch(ArithmeticException e) {
      System.out.println("Cannot divide by zero");
    }
    finally {
      System.out.println("Program Completed");
    }
  }
}
```

Output

```
Cannot divide by zero
Program Completed
```

Step-by-step trace

1. The `try` block starts. `10/0` raises an `ArithmeticException`.
2. `System.out.println(result);` inside `try` is skipped. Control jumps straight out of `try`.
3. Java looks for a matching `catch`. `catch(ArithmeticException e)` matches, so `Cannot divide by zero` is printed. The variable `e` is the reference to the exception object.
4. The `finally` block runs and prints `Program Completed`.

If the division had been `10/2`, the output would be `5` followed by `Program Completed`. The `catch` block would be skipped but `finally` would still run.

> **Pro-Tip:** In assignments and exams on exception handling, the expected structure is `try` + `catch` + `finally`. Always include `finally`, and remember it runs with or without an exception.

7. Multiple Catch Blocks

A single `try` block can contain several statements that raise **different** exceptions. For that we attach multiple `catch` blocks to one `try`. Each `catch` handles one exception type.

```
class Demo {
  public static void main(String args[]) {
    try {
      int arr[] = new int[5];
      arr[10] = 100;
      int x = 10/0;
    }
    catch(ArrayIndexOutOfBoundsException e) {
      System.out.println("Invalid array index");
    }
    catch(ArithmeticException e) {
      System.out.println("Division error");
    }
  }
}
```

- The array has size 5 (valid indices `0` to `4`), but we write to `arr[10]`, which gives an `ArrayIndexOutOfBoundsException`.
- `10/0` would give an `ArithmeticException`.
- The first `catch` handles the array error and the second handles the arithmetic error.

The same idea extends to three, four or more `catch` blocks: one catch per exception type.

> **Note for Review:** Only **one** catch block runs for a single exception. In the program above, the exception at `arr[10] = 100;` jumps out of `try` immediately, so `10/0` is **never reached**. The output is only `Invalid array index`. To see `Division error` you would have to trigger it separately (for example by removing the array line). Also, when exception types are related by inheritance, place the **more specific** catch before the **more general** one (for example `ArithmeticException` before `Exception`), otherwise the code will not compile.

8. Nested try Blocks

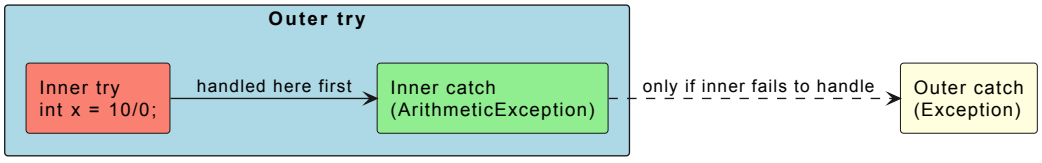
A **try block inside another try block** is a nested try.

```
class Demo {
  public static void main(String args[]) {
    try {
      try {
        int x = 10/0;
      }
      catch(ArithmeticException e) {
        System.out.println("Inner Exception");
      }
    }
    catch(Exception e) {
      System.out.println("Outer Exception");
    }
  }
}
```

- The inner `try` has its own inner `catch(ArithmeticException e)`, which handles the known error `10/0`.
- The outer `try` has its own outer `catch(Exception e)`. Because the outer one catches the general `Exception`, it acts as a safety net for anything unexpected that is not handled by the inner block.

Output

```
Inner Exception
```



The rule is that a thrown exception is offered to the nearest enclosing `catch` first. If that block cannot handle it, the exception moves outward to the next `catch`. Each `try` needs its own `catch` (or `finally`).

9. The throw Keyword

So far Java raised exceptions *for* us. The `throw` keyword lets **us** raise an exception explicitly.

Syntax

```
throw new ExceptionType();
```

Example: age eligibility

```
class Demo {
  static void checkAge(int age) {
    if(age < 18)
      throw new ArithmeticException("Not eligible");
    else
      System.out.println("Eligible");
  }

  public static void main(String args[]) {
    checkAge(15);
  }
}
```

Output

```
Exception in thread "main" java.lang.ArithmeticException: Not eligible
```

Explanation:

- `checkAge(15)` is called, and since `15 < 18` the `if` condition is true.
- The program creates an `ArithmeticException` object carrying the message `Not eligible` and throws it.
- Nothing catches it, so the program terminates and prints the standard message.
- If we called `checkAge(20)`, the condition is false, the `else` branch runs, and the output is simply `Eligible`.

The same pattern fits any eligibility rule: voting age, minimum balance, minimum marks, and so on. (`static` is used so `main` can call `checkAge` without creating an object.)

10. The throws Keyword

The `throws` keyword **declares the exceptions that a method may generate**. It is a warning label on the method signature that tells callers "this method might fail in this way, so be ready."

```
import java.io.*;

class Demo {
  static void readFile() throws IOException {
    FileReader f = new FileReader("data.txt");
  }

  public static void main(String args[]) {
    try {
      readFile();
    }
    catch(IOException e) {
      System.out.println("File Error");
    }
  }
}
```

- `readFile()` does not handle the problem itself. It declares `throws IOException`.
- `main` calls `readFile()` inside a `try` block and handles the problem in `catch(IOException e)`.
- If `data.txt` cannot be read, the output is `File Error`.

The responsibility for the exception is passed from the method to its caller. This is the other legal option for a checked exception: **handle it with try-catch, or declare it with throws**.

11. throw vs throws

`throw`	`throws`
Used to explicitly throw an exception	Used to declare exceptions
Used inside the method body	Used with the method signature
Throws one exception	Can declare multiple exceptions

Example of declaring multiple exceptions:

```
void test() throws IOException, SQLException {
  // method body
}
```

> **Pro-Tip:** A memory trick: **throw** is the *action* (one throw at a time, in the body). **throws** is the *announcement* (a list, in the signature).

12. User-Defined (Custom) Exceptions

Java already provides many exceptions, but your application may have its own kind of problem, such as an invalid age or an insufficient balance. Java allows programmers to create custom exceptions **by extending the `Exception` class**. The name is up to you and should reflect your requirement.

Example: InvalidAgeException

```
class InvalidAgeException extends Exception {
  InvalidAgeException(String message) {
    super(message);
  }
}

class Voting {
  static void checkAge(int age) throws InvalidAgeException {
    if(age < 18)
      throw new InvalidAgeException("Not eligible for voting");
    else
      System.out.println("Eligible for voting");
  }

  public static void main(String args[]) {
    try {
      checkAge(16);
    }
    catch(InvalidAgeException e) {
      System.out.println(e.getMessage());
    }
  }
}
```

Output

```
Not eligible for voting
```

How the Pieces Fit Together

1. **Create the exception class.** `InvalidAgeException extends Exception` makes it a new kind of checked exception. Its constructor takes a `String message` and passes it to the parent with `super(message)`.
2. **Declare it.** `checkAge(int age) throws InvalidAgeException` warns callers that this method can raise it.
3. **Throw it.** If `age < 18` the method executes `throw new InvalidAgeException("Not eligible for voting")`.
4. **Catch it.** `main` calls `checkAge(16)` inside `try`. The `throw` fires, control jumps to `catch(InvalidAgeException e)`, and `e.getMessage()` returns the stored text, which is printed.



> **Pro-Tip:** The recipe for *any* custom exception is the same three parts: (1) a class that extends `Exception` with a message constructor, (2) a method that declares `throws` and uses `throw` under some condition, (3) a caller with `try` / `catch`. Only the name and the condition change.

13. Summary

After completing this module, students should be able to:

- Identify checked, unchecked, and error types in Java.
- Handle runtime errors using exception handling mechanisms.
- Use `try`, `catch`, `finally`, `throw`, and `throws`.
- Create custom exception classes.

14. Practice Exercises

Write the **complete** program for each (imports if needed, class, `main`, and the blocks), then compile it with `javac` and run it with `java`. A free JDK (Java Development Kit) is enough to run everything. You may write the logic directly inside `main` or in separate methods that `main` calls.

1. **Write a Java program to handle division by zero using exception handling.** *Hint:* `try` holds the division, `catch(ArithmeticException e)` prints a message, and `finally` prints a completion message.

2. **Create a program that handles invalid array indexing exceptions.** *Hint:* Same structure as above, with `ArrayIndexOutOfBoundsException`.

3. **Implement multiple catch blocks for different exception types.** *Hint:* Put two or three risky statements in one `try` and give each exception type its own `catch` (for example catch 1 for the array index, catch 2 for division by zero, catch 3 for a third type). `finally` still runs whether or not an exception occurs.

4. **Create a user-defined exception `InsufficientBalanceException` for a banking application.** *Hint:* Follow the `InvalidAgeException` pattern exactly:

   - Create `InsufficientBalanceException extends Exception` with a message constructor.
   - Create a banking method that takes the **balance** (rather than the age) as its parameter and declares `throws InsufficientBalanceException`.
   - Choose a threshold (for example 5000). If the balance is below it, `throw` the exception, otherwise print that the balance is sufficient.
   - In `main`, call the method in a `try` block and print `e.getMessage()` in the `catch` block.

15. Think-It-Through Questions

1. In the program below, predict the exact output and explain, step by step, why each line appears (or does not appear). Then explain what would change if the `catch` were moved to a `catch(Exception e)` placed *before* the `ArithmeticException` catch.

```
try {
  System.out.println("A");
  int x = 10 / 0;
  System.out.println("B");
}
catch(ArithmeticException e) {
  System.out.println("C");
}
finally {
  System.out.println("D");
}
System.out.println("E");
```

2. A teammate says: "Our `readFile()` method declares `throws IOException`, so exceptions in it can never crash the program." Is that correct? Describe exactly who is responsible for the exception after the declaration, and what happens if nobody in the call chain catches it. Does `finally` still run in that scenario?

3. `ArrayIndexOutOfBoundsException` and `FileNotFoundException` are both exceptions, yet the compiler treats them completely differently. Explain the reason for this design choice. What would programming be like if *every* exception were checked, and what if *none* were?

4. You are designing the `InsufficientBalanceException` for a bank. Should it extend `Exception` or `RuntimeException`? Argue for one choice by describing what the compiler will force callers to do, and what could go wrong in a real banking system under each choice.

5. In a nested `try` structure, the inner `catch` handles `ArithmeticException` and the outer `catch` handles `Exception`. Suppose the inner `try` instead throws an `ArrayIndexOutOfBoundsException`. Trace which `catch` blocks run, in what order, and whether any `finally` blocks (one on each level) execute before the program continues. Then explain why a `StackOverflowError` is a poor candidate for the same safety net.